武汉大学国家网络安全学院

实验报告

课程名称：舆情分析

专业年级：2021 级网络空间安全

程子洋、邵嘉阳、王卓、彭子恒、刘虓 小组

姓　　名：程子洋

学　　号：2021301051114

协 作 者：邵嘉阳、王卓、彭子恒、刘虓

实验学期：2023-2024 学年　　第 1 学期

课堂时数：32 课时　　课外时数：无

填写时间：2023 年 12 月 13 日

目录

一、实验介绍 **1**
1.1 实验名称: 综合大实验之话题-观点图谱 1
1.2 实验目的 1
1.3 实验环境 1
1.4 参考文献 2

二、实验内容 **3**
2.1 研究内容 3
2.1.1 实体识别 3
2.1.2 关系抽取 3
2.1.3 观点抽取 4
2.2 实验方案设计 4
2.2.1 数据集收集 4
2.2.2 数据集标注 7
2.2.3 数据预处理 10
2.2.4 模型介绍 18
2.2.5 网页展示 31
2.3 评估方案 36
2.3.1 Casrel 36
2.3.2 NER_IDCNN 37
2.3.3 RE_BGRU_2ATT 38
2.4 实验成果 40
2.5 实验结果分析 41
2.6 创新性说明 43
2.6.1 CasRel 43
2.6.2 NER_IDCNN 43
2.6.3 RE_BGRU_2ATT 43
2.6.4 PL-maker 44
2.7 实验总结 45

一、 实验介绍

1.1 实验名称: 综合大实验之话题-观点图谱

1.2 实验目的

本任务旨在通过数据采集、信息可视化展示、模型搭建以及实验报告的方式，构建一个话题-观点图谱。

1. 数据采集：针对亚太组织领域，通过采集推特和新闻平台（BBC, CNN, New York Times, NBC News, The Guardian）等来源的数据，收集与关键词（新亚太联盟、美日印澳、美日联盟、五眼联盟、美日韩、东南亚联盟、东盟）相关的内容。
2. 数据信息可视化展示：展示采集到的数据内容，包括文本样本的数量、来源平台等信息。通过可视化方式直观地呈现数据的特征。
3. 模型搭建：设计并搭建一个能够从文本中抽取实体、关系、观点的模型。模型的任务包括实体抽取、关系抽取和观点抽取，以构建话题-观点图谱。
4. 网页展示分析效果：利用网页展示模型分析的结果，包括抽取的实体、关系、观点等信息，并以图谱的形式展示。用户可以通过交互方式查看分析结果。
5. 掌握舆情分析关键技术：识别文本中的实体，例如人物、地点、组织，以理解舆情中涉及的关键角色。使用命名实体识别技术，训练模型以准确识别并分类文本中的命名实体。识别文本中实体之间的关系，观点，这有助于构建实体关系图，进一步分析舆情中不同实体之间的关联。

1.3 实验环境

1. 硬件:
   (a) 设备名称：LAPTOP-1MSR40R8
   (b) 处理器：12th Gen Intel(R) Core(TM) i5-12450H 2.00 GHz
   (c) 机带：RAM 16.00 GB
   (d) 显卡：NVIDIA Geforce RTX 3050 Laptop GPU
2. 软件:PyCharm 2023.2
3. 解释器版本与所需库: 具体环境配置要求见mumuyeye/Public-Opinion-Analysis: 武汉大学 2021级网安舆情分析大作业 (github.com) 中的 readme 文件，下面只列举所需的关键库。
   (a) NER_IDCNN_CRF 与 RE_BGRU_2ATT：python3.6 + tensorflow1.14.0 + scikit_learn + jieba
   (b) CasRel：python3.7 + tensorflow-gpu1.13.1 + cuda10.0 + keras-bert0.80.0 + jieba

1.4 参考文献

1. Attention-Based Bidirectional Long Short-Term Memory Networks for Relation Classification
2. Neural Relation Extraction with Selective Attention over Instances
3. A Novel Cascade Binary Tagging Framework for Relational Triple Extraction
4. Fast and Accurate Entity Recognition with Iterated Dilated Convolutions
5. Packed Levitated Marker for Entity and Relation Extraction
6. < 关系抽取 >ACL2022 关系抽取 SOTA 之 PL-Marker

二、 实验内容

2.1 研究内容

根据用户的生成的内容，抽取其中涉及的实体、观点，根据抽取的内容构建话题-观点图谱。例如图 1 中是用户‘杨建利’发表在推特的内容，我们可以通过实体抽取、关系抽取、观点抽取、以及观点推理得到图 2 的内容。

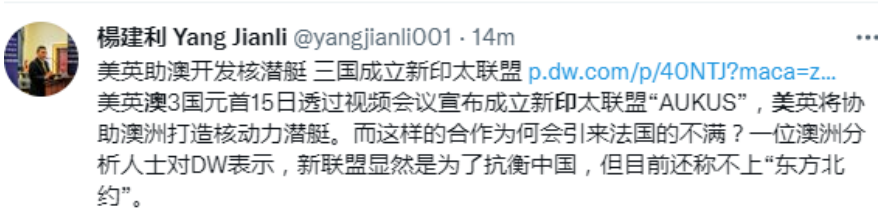


图 1: 内容

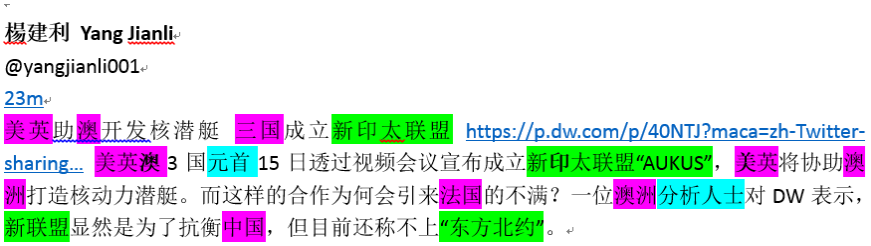


图 2: 抽取

2.1.1 实体识别

实体识别是自然语言处理中的一个重要任务，其目标是从文本中识别出具有特定意义的实体。实体可以是具体的人、地点、组织等，也可以是抽象的概念、时间、日期等。实体识别技术可以帮助我们理解文本中具体指称的含义，为后续的信息抽取和语义理解任务提供基础。本次实验主要关注的实体是国家、机构和人物。示例如下：

- **国家**：美、英、澳、法国、中国
- **人物**：元首、人士
- **机构**：新印太联盟、新印太联盟“AUKUS”东方北约

2.1.2 关系抽取

关系抽取是指从文本中提取出实体之间的关系。这些关系描述了实体之间的联系、依存或关联。关系抽取可以帮助我们发现实体间的关联信息，从而构建出知识图谱或网络。这对于知识管理、信息提取和问答系统等应用具有重要意义。本次关系抽取的目标是二元关系，具体定义如下：

- **预定义关系**：国家间、机构间、国家-机构、机构-人物、国家-人物
- **抽取结果**：< 美，英，国家间 >，< 美，元首，国家-人物 >，< 美，新印太联盟，国家-机构 >，…

2.1.3 观点抽取

观点抽取是从文本中提取出包含主观性和情感色彩的观点或意见。观点抽取可以帮助我们了解人们对特定主题的看法、态度和情感倾向。在社交媒体分析、产品推荐和舆情监测等领域，观点抽取被广泛应用。本次实验的观点定义如下：

- **已知观点**：< 美，澳，+>，< 英，澳，+>，< 法，新亚太联盟，->，< 新亚太联盟，中国，->
- **推理观点**：< 美，中国，->, < 英，中国，->，< 澳洲，中国，->，< 法国，中国,？ >

2.2 实验方案设计

2.2.1 数据集收集

因没有符合任务目标的数据，我们需要人为收集数据集。基于任务特点，我们通过撰写爬虫代码以及基于*EasySpider*工具，采集亚太组织领域的数据，关键词包括新亚太联盟、美日印澳、美日联盟、五眼联盟、美日韩、东南亚联盟、东盟。使用推特和新闻平台（BBC, CNN, New York Times, NBC News, The Guardian）进行数据采集，共采集 2000 篇左右的数据以供后续数据集的处理与搭建。

1. 推特数据的爬取

对于推特数据的爬取，我们使用了一个名为 Scweet 的 Python 库，该库专门用于从 Twitter 上爬取数据，模拟用户在 Twitter 上的搜索行为以使用户能够按关键词、时间范围等条件来搜索和获取推文数据。

```
from Scweet.scweet import scrape
from Scweet.user import get_user_information, get_users_following,
    get_users_followers

data = scrape(words=['新亚太联盟', '美日印澳', '美日联盟', '五眼联盟',
    '美日韩', '东南亚联盟', '东盟', '亚太组织',
                '新亚太联盟成员', '亚太合作', '新印太联盟','东方北约']
, since="2019-12-08", until="2019-12-31", from_account = None,interval=1,
    headless=False, display_type="Top", save_images=False, proxy = None,
        save_dir = 'outputs',
         resume=False, filter_replies=True, proximity=False,lang=None)
```

下面是该爬虫代码的参数介绍：

- **words**：关键词列表，用于搜索相关推文。
- **since 和 until**：指定时间范围，筛选在指定时间段内的推文。
- **interval**：设置请求之间的时间间隔，以防止被限制或封锁。
- **display_type**：设置获取的推文类型，此处为“Top”表示热门推文。
- **filter_replies**：是否过滤掉回复推文。
- **proximity**：是否按照时间接近性进行筛选。

- **lang**：指定语言。

下面是爬取数据的部分展示：



图 3: 爬取的原始数据

2.New York Times & BBC 数据采取

对于 New York Times 和 BBC News 的数据的爬取，考虑到网站较好的反爬取机制，通过编写代码较难实现数据的获取。这里我们使用了*EasySpider*工具，该工具可以使用图形化界面，无代码可视化的设计和执行任务。只需要在网页上选择自己想要操作的内容并根据提示框操作即可完成任务的设计和执行。同时软件还可以单独以命令行的方式进行执行，从而方便的嵌入到其他系统中。具体操作可见mumuyeye/Public-Opinion-Analysis: 武汉大学 2021 级网安舆情分析大作业 (github.com)

下面是基于该工具爬取到的数据部分展示 (考虑篇幅原因，仅展示爬取的纽约时报的原始数据)：

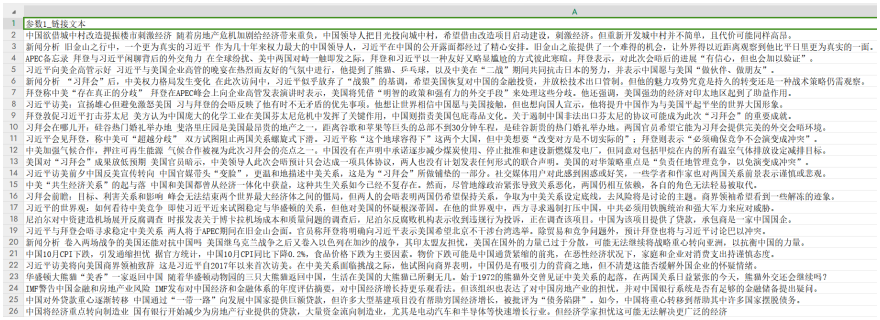


图 4: 纽约时报原始数据 (部分)

我们对爬取的数据进行可视化分析，清晰展示任务所使用的数据。通过图表、词云等可视化手段，我们将呈现关键词的出现频率、时间趋势，以及可能的关联性。这样的可视化分析不仅有助于直观地理解数据的特征，还能为后续的观点图谱构建提供更全面的基础。

下面是对数据的词云图分析：



图 5: 词云图

下面是对数据的词频分析条形图：

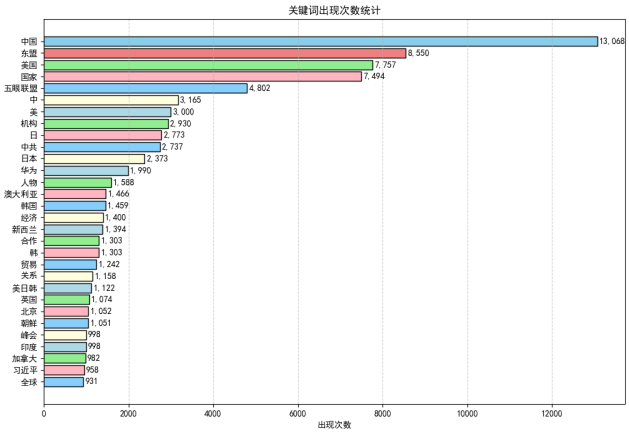


图 6: 词频分析条形图

“中国”是出现次数最多的关键词，表明在所爬取的数据中，中国是一个重要的关注点。“东盟”、“美国”、和“国家”也是高频关键词，反映了对于东盟、美国以及国家概念的广泛讨论。“五眼联盟”以及“美日韩”出现次数较高，表明数据集大部分涉及到国际组织、联盟关系和区域合作的话题。

“经济”、“贸易”和“合作”等词语的高频出现，表明数据集涉及到国际经济合作和贸易关系的讨论。

2.2.2 数据集标注

本组综合考虑后，选择采用 label-studio 工具进行标注，具体流程图下：

1. 安装:

```
pip install label-studio
```

2. 在终端启动:

```
label-studio start //打开命令行，直接输入
```

3. 创建项目点击 create（第一次使用需要创建账号）:

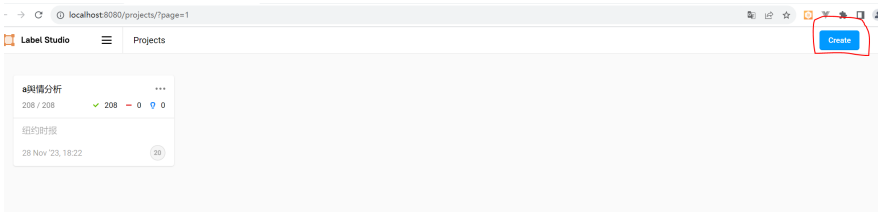


图 7: 创建项目

4. 设置基本信息后上传文件（经过本组实践，建议选择 txt 类型文件）:

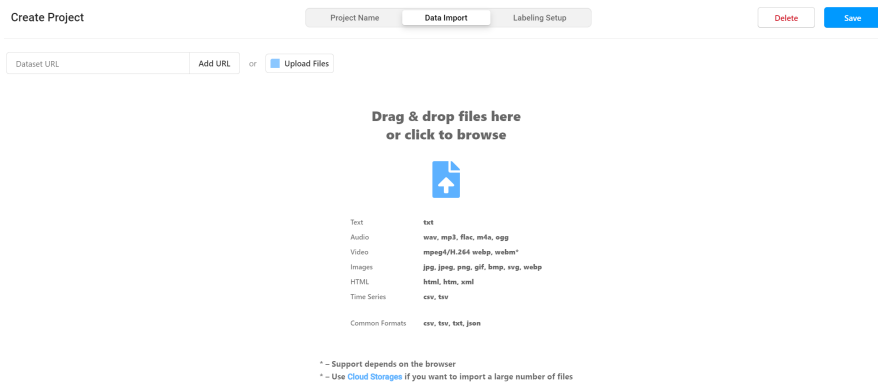


图 8: 上传文件

5. 选择自然语言处理-命名实体识别：

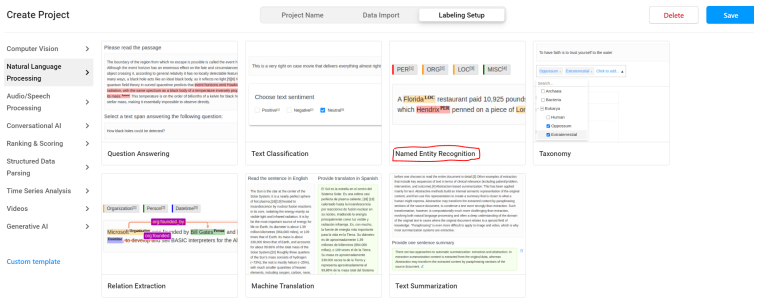


图 9: 选择类型

6. 按照自己项目的需求设置标签：

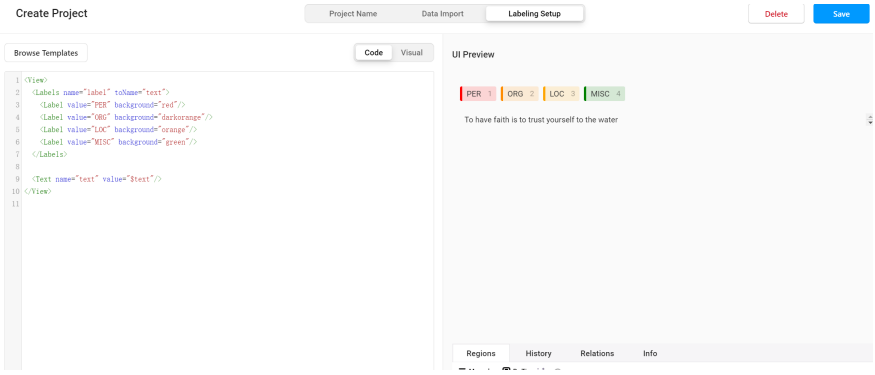
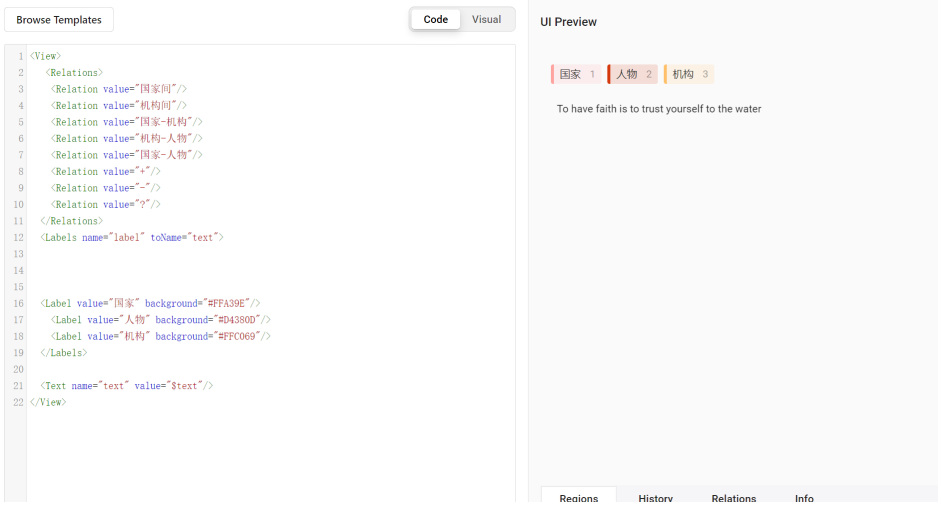


图 10: 设置标签

本组的标签设置如下：




```
<View>
  <Relations>
    <Relation value="国家间"/>
    <Relation value="机构间"/>
    <Relation value="国家-机构"/>
    <Relation value="机构-人物"/>
    <Relation value="国家-人物"/>
    <Relation value="+"/>
    <Relation value="-"/>
    <Relation value="?"/>
  </Relations>
  <Labels name="label" toName="text">


    <Label value="国家" background="#FFA39E"/>
    <Label value="人物" background="#D4380D"/>
    <Label value="机构" background="#FFC069"/>
  </Labels>

  <Text name="text" value="$text"/>
</View>
```

图 11: 示例标签

7. 数据标注（示例）：

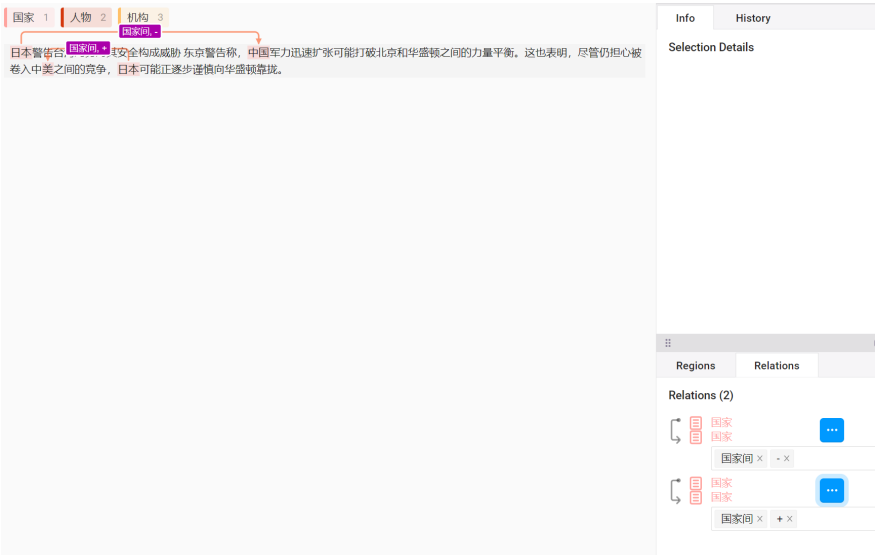


图 12: 数据标注

8. 完成数据标注后，可以在以下选项中导出项目所需要的格式：

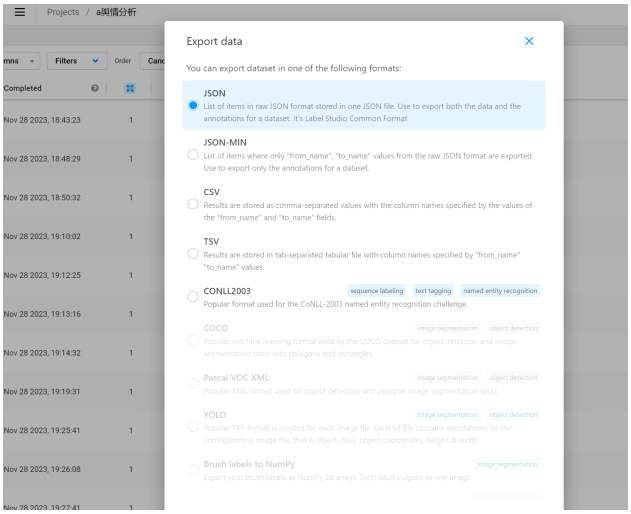


图 13: 导出数据

此外，为了使得不同成员对数据标注的风格更为接近，我们规定了统一的标注标准。

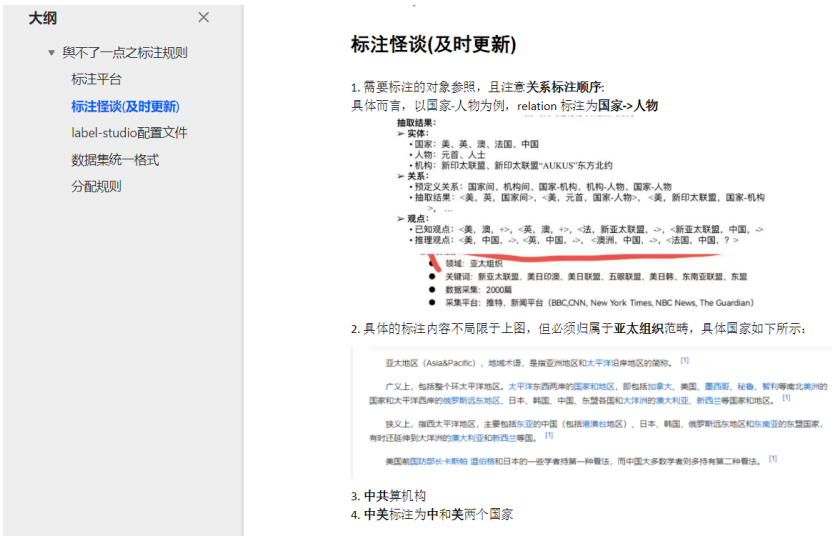


图 14: 数据标注标准

2.2.3 数据预处理

基于 label-studio 标注导出的数据，我们针对不同模型训练所需的不同数据格式撰写对应的数据预处理代码。下面将分别对不同模型使用的数据预处理流程进行介绍：

1. CasRel:

   (a) CasRel 数据格式

   CasRel 所需的数据格式是一个列表，列表以字典作为列表中的元素，每一个字典代表一条数据。字典中包含两个字段："text" 和 "triple_list"。"text" 字段中存放的是此条数据分词后的文本信息。"triple_list" 字段中存放的是从文本信息中解析出的关系，每一条关系以 ["实体 S" ，"关系 R"，"实体 O"] 这样的三元组表示。

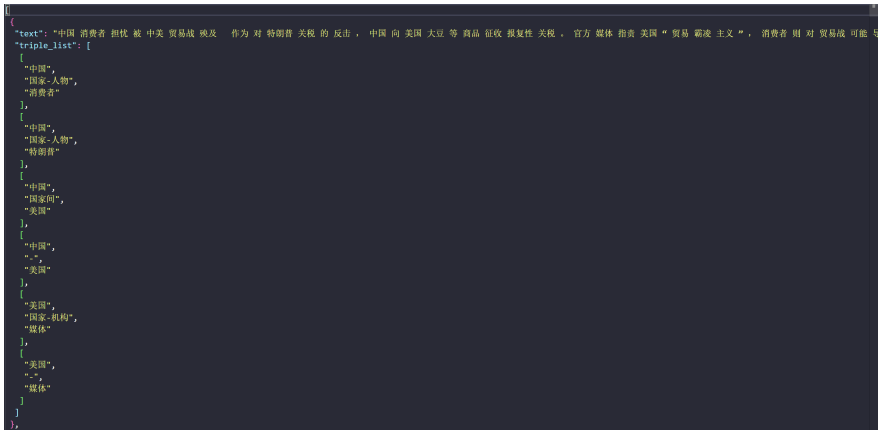


图 15: CasRel 数据格式

(b) 数据格式转换

数据格式转换的方式以注释的形式在代码中进行介绍。

```
def convert_format(data_list):
    converted_data_list = []

    for data in data_list: # 逐条提取数据
        sentences = [' '.join(jieba.cut(sentence, cut_all=False)) for sentence
            in data["data"]["text"].split("\n") if
                sentence.strip()] # 对文本进行分词

        ner = [] # 定义实体
        relations = [] # 定义关系

        entity_map = {}

        rows = 1000
        cols = 1000
        array_2d = [['' for j in range(cols)] for i in range(rows)] #
            构建单词表，方便找到对应单词

        for annotation in data["annotations"]:
            ner_annotation = []
            for entity in annotation["result"]:
                if "value" in entity and "start" in entity["value"]:
                    entity_id = entity["id"]
                    start = entity["value"]["start"]
                    end = entity["value"]["end"]
                    label = entity["value"]["labels"][0]
                    entity_map[entity_id] = {"start": start, "end": end}
                    array_2d[start][end] = entity["value"]["text"] # 构建单词表
                    ner_annotation.append([start, end, label])
            ner.append(ner_annotation)

            for relation in annotation["result"]:
                if "relation" in relation["type"]:
                    relation_type = relation["labels"]
                    from_id = relation["from_id"]
                    to_id = relation["to_id"]

                    if from_id in entity_map and to_id in entity_map:
                        from_start = entity_map[from_id]["start"]
                        from_end = entity_map[from_id]["end"]
```

```
                to_start = entity_map[to_id]["start"]
                to_end = entity_map[to_id]["end"]
            # 生成“triple_list”中的关系列表
            if len(relation_type) == 1:
                if relation_type[0] == '?':
                    relations.append([array_2d[from_start][from_end], "?",
                        array_2d[to_start][to_end]])
                else:
                    relations.append([array_2d[from_start][from_end],
                        relation_type[0], array_2d[to_start][to_end]])
            elif len(relation_type) == 2:
                if relation_type[0] == '?':
                    relations.append([array_2d[from_start][from_end], "?",
                        array_2d[to_start][to_end]])
                else:
                    relations.append([array_2d[from_start][from_end],
                        relation_type[0], array_2d[to_start][to_end]])
                if relation_type[1] == '?':
                    relations.append([array_2d[from_start][from_end], "?",
                        array_2d[to_start][to_end]])
                else:
                    relations.append([array_2d[from_start][from_end],
                        relation_type[1], array_2d[to_start][to_end]])

        # 将分词后的句子连接成一个字符串
        sentences = ' '.join(sentences)

        converted_data = {
            "text": sentences,
            "triple_list": relations
        }

        converted_data_list.append(converted_data)

    return converted_data_list
```

2. NER_IDCNN:

   (a) NER_IDCNN 数据格式

   NER_IDCNN 所需的数据格式是使用 BIO 标注方法标注的序列文本，如下图所示将每个词（或字符）标注为三个类别之一：B（Beginning）、I（Inside）、O（Outside），分别表示为一个实体的开始部分、一个实体的中间部分、一个实体的结束部分。

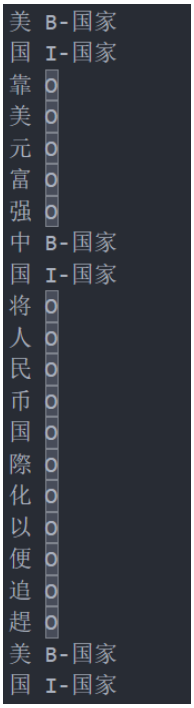


图 16: NER_IDCNN 数据格式

(b) 数据格式转换

我们遍历数据集中的每一行，获取文本和对应的标签信息。初始化 label_list 列表，其中每个元素对应于文本中的一个字符，初始值为'O'，表示不属于任何实体。解析标签信息，更新 label_list，将命名实体用 BIO 标签标注在对应位置。将文本和标签写入保存路径，每个字符及其标签占一行，字符和标签之间用空格分隔，每个文本样本之间用空行分隔。

```
def gen_train_data(file_path, save_path):
    """
    file_path: 通过Label Studio导出的csv文件
    save_path: 保存的路径
    """
    data = pd.read_csv(file_path,encoding='utf-8')
    for idx, item in data.iterrows():
        text = item['text']
        if pd.isna(text):
            text = ''
        text_list = list(text)
        label_list = ['O' for _ in range(len(text_list))]

        labels = item['label']
        if not pd.isna(labels):
            labels = json.loads(labels)
            for label_item in labels:
```

```
            start = label_item['start']
            end = label_item['end']
            label = label_item['labels'][0]

            # Update label_list based on BIO format
            label_list[start] = f'B-{label}'
            label_list[start + 1:end] = [f'I-{label}' for _ in range(start
                + 1, end)]

    assert len(label_list) == len(text_list)

    with open(save_path, 'a',encoding='utf-8') as f:
        for idx_, (char, label) in enumerate(zip(text_list, label_list)):
            if char == '\t' or char == ' ':
                char = ','
            line = f"{char} {label}\n"
            f.write(line)
        f.write('\n')
```

3. RE_BGRU_2ATT

(a) RE_BGRU_2ATT 数据格式

根据实验要求，我们事先在 relation2id.txt 中定义好了 8 种预定义关系类型，分别是国家间、机构间、国家-机构、机构-人物、国家-人物，以及它们相对应的观点立场："+" 表示正面立场，"-" 表示负面立场，"?" 表示中立或未知，并分别用数字进行标记标号，如下图所示。

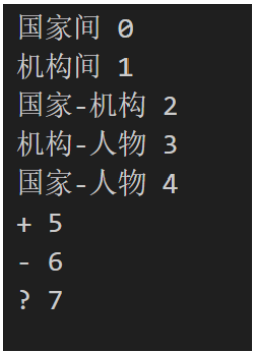


图 17: RE_BGRU_2ATT 预定义关系类型

RE_BGRU_2ATT 需要数据用来进行关系抽取，因此数据集中每一行的数据格式是：
" 实体 1、" 实体 2"、" 关系类型"、" 文本内容"
如下图所示中，文本内容一般是一句话，在我们的实验中就是所爬取到的新闻内容，而实体 1 和实体 2 则分别为新闻文本内容中所出现的主体，并用事先定义好的关系类型进行描述。

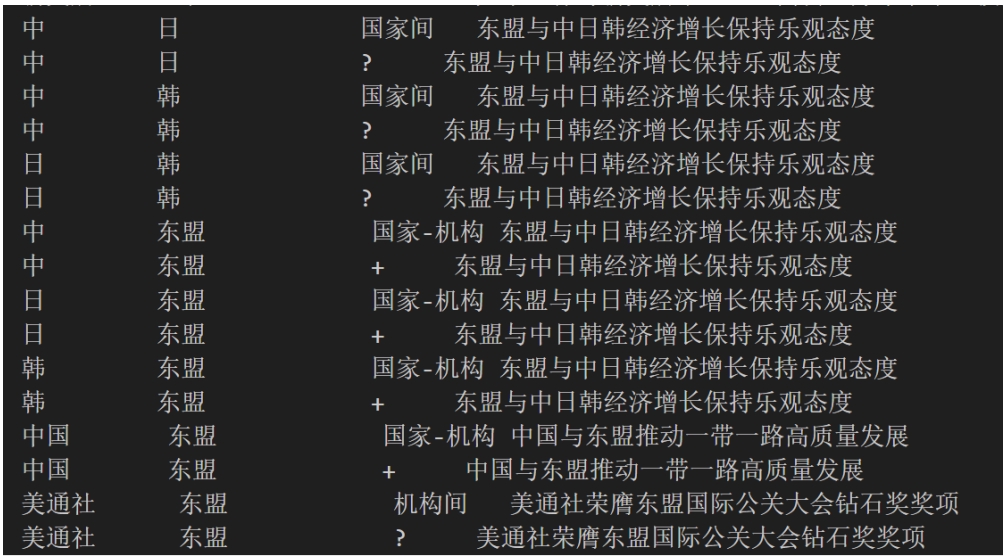


图 18: RE_BGRU_2ATT 数据格式

(b) 数据格式转换

我们遍历数据集中的每一行，获取文本和对应的标签信息。检查 annotation 的类型是否为'relation'。如果是关系类型，则从注释中提取实体 1id 和实体 2id。使用实体 1id 和实体 2id 在注释中查找相应的实体文本（entity1 和 entity2）。然后从注释中获取关系类型，假设每个关系只有一个标签，提取第一个标签。接着从提供的数据中获取原始句子文本。最后我们将获取的" 实体 1、" 实体 2"、" 关系类型"、" 文本内容" 数据进行保存。

```
def extract_relationships(data):
    relationships = []

    for annotation_task in data:
        text = annotation_task['data']['text']
        annotations = annotation_task['annotations'][0]['result']

        for annotation in annotations:
            if annotation['type'] == 'relation':
                entity1_id = annotation.get('from_id')
                entity2_id = annotation.get('to_id')

                entity1 = next((v['value']['text'] for v in annotations if
                    v['id'] == entity1_id), None)
                entity2 = next((v['value']['text'] for v in annotations if
                    v['id'] == entity2_id), None)

                relationship_type = annotation.get('labels', [])[0] # Assuming
                    each relation has a single label

```

```
            # Use the original text provided in the data
            sentence = text.strip()

            relationships.append({
                'entity1': entity1,
                'entity2': entity2,
                'relationship_type': relationship_type,
                'sentence': sentence
            })

    return relationships
```

4. PL-Marker(Packed Levitated Markers)

   (a) PL-Marker 数据格式

   PL-Marker 所需的数据格式是一个字典，每个字典包含三个键值：sentences、ner 以及 relations。“sentences” 代表每条数据，每条数据都是由经过分字之后组成的列表。“ner” 包含每条数据中实体的边界位置和实体类型的列表，比如，“[26，26，“国家”]” 表示该数据第 26 个位置的字表示一个表示一个国家实体。“relations” 包含每条数据中关系的信息，包括两个实体的边界位置和关系类型。

```
{
  # sentences in the document, each sentence is a list of tokens
  "sentences":
    ["中", "国", "共", "产", "党", ...],

  # entities (boundaries and entity type) in each sentence
  "ner": [
    [...],
    [...],
    [[26, 26, "国家"], ...], #the boundary positions are indexed in the
        document level
    ...,
  ],

  # relations (two spans and relation type) in each sentence
  "relations": [
    [...],
    [...],
    [[14, 14, 10, 10, "国家间"], ...],
    ...
  ]
}
```

(b) 数据格式转换

我们定义了一个函数 convert_format, 首先创建一个空列表 converted_data_list，用于存储转换后的数据。对输入的每个数据进行遍历，其中每个数据包含了原始文本数据以及与之相关的标注信息。然后从每个数据中提取句子信息，实体的起始位置、结束位置和标签，关系标签，分别存储在 sentences、ner 和 relations 变量中。最后将每个数据的转换结果添加到 converted_data_list 中。

```
def convert_format(data_list):
    converted_data_list = []

    for data in data_list:
        sentences = list(data["data"]["text"])
        ner = []
        relations = []
        entity_map ={}

        for annotation in data["annotations"]:
            ner_annotation = []
            for entity in annotation["result"]:
                if "value" in entity and "start" in entity["value"]:
                    entity_id = entity["id"]
                    start = entity["value"]["start"]
                    end = entity["value"]["end"] - 1
                    label = entity["value"]["labels"][0]
                    entity_map[entity_id] = {"start": start, "end": end}
                    ner_annotation.append([start, end, label])
            ner.append(ner_annotation)

            relation_annotation = []
            for relation in annotation["result"]:
                if "relation" in relation["type"]:
                        relation_labels = relation.get("labels", [])
                        from_id = relation["from_id"]
                        to_id = relation["to_id"]

                        if from_id in entity_map and to_id in entity_map:
                            for relation_label in relation_labels:
                                from_start = entity_map[from_id]["start"]
                                from_end = entity_map[from_id]["end"]
                                to_start = entity_map[to_id]["start"]
                                to_end = entity_map[to_id]["end"]
                                relation_annotation.append([from_start, from_end,
                                    to_start, to_end, relation_label])
```

```
            relations.append(relation_annotation)

        converted_data = {
            "sentences": sentences,
            "ner": ner,
            "relations": relations,
        }

        converted_data_list.append(converted_data)

    return converted_data_list
```

2.2.4 模型介绍

1. **CasRel**

(a) 三元组重叠问题

三元组重叠问题也称实体关系重叠问题，关注的是当输入文本中有多个实体关系三元组时，彼此之间可能共享了某些实体。下图列举了三种形式，具体说明了三元组重叠问题。

i. **Normal**: 代表没有重叠的部分。

ii. **EPO(EntityPairOverlap)**: 关系两端的实体都是一致的，例如 QT 既在电影 DU 中扮演角色，又是电影 DU 的执导。

iii. **SEO(SingleEntityOverlap)**: 关系两端只有单个实体共享，图中的例子，从小范围来说，JRB 出生在 Washington, 但是 Washington 是 USA 的首都，所以也可以说 JRB 出生在 USA。

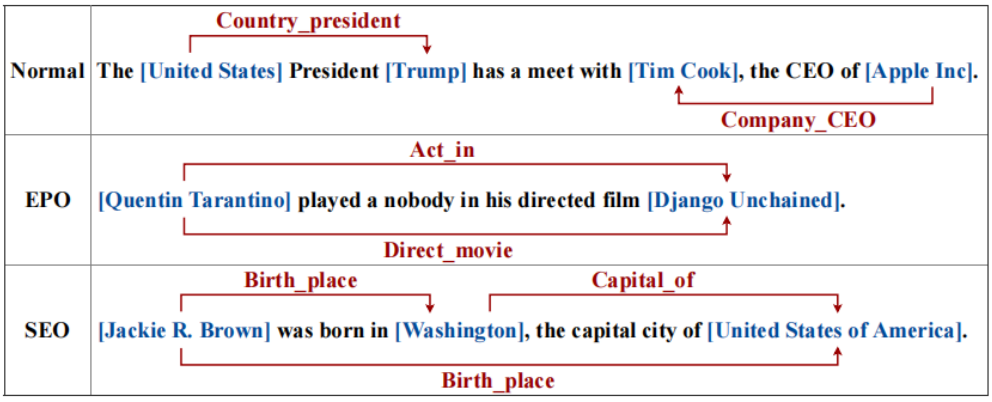


图 19: 三元组重叠问题

在我们的标注数据中也有许多存在三元组重叠问题的文本。

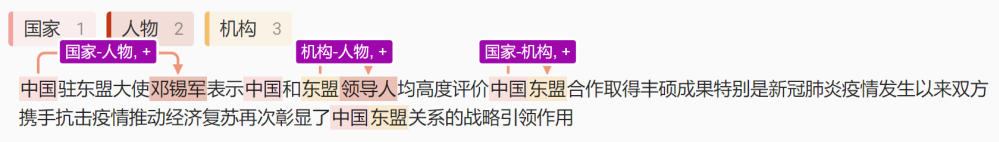


图 20: 标注数据中的三元组重叠问题

(b) 核心思想

以往的思路是把关系抽取任务当做一个多分类的任务来做 (1)，S 和 O 分别代表主语和宾语实体，预测这两个实体之间的关系，联合解码模型就是按照此思想解决关系抽取任务的。但联合解码模型由于其标签设计或 CRF 层限定了每个 token 只能有一个 tag，所以无法适用于实体关系重叠情况。此外，基于参数共享的关系抽取方法最后通常是一个多分类层，也就是一对实体只能有一个标签。简单地将其改成多标签分类就能一定程度上解决实体关系重叠问题，但这种改进并不具备什么创新性。

$$f(S, O) \rightarrow R \tag{1}$$

CasRel 对关系抽取任务进行了全新的分解 (2)，根据主语实体 S 和关系 R，预测宾语实体 O。这种思想解决了传统关系抽取任务所面对的三元组重叠问题，把多标签的部分当作了一个 function(f_R)。

$$f_R(S) \rightarrow O \tag{2}$$

(c) 模型流程

CasRel 抽取关系三元组主要包括两个步骤:

i. 识别出文本中的实体 (S)。

ii. 根据识别出的所有实体 (S)，识别出所有可能的关系 (R) 和实体 (O)。

(d) 模型框架

CasRel 模型以 BERT 编码层模块 (BERT Encoder)、主体识别模块 (Subject Tagger) 和特定关系下的课题识别模块 (Relation-specific Object Taggers) 组成。

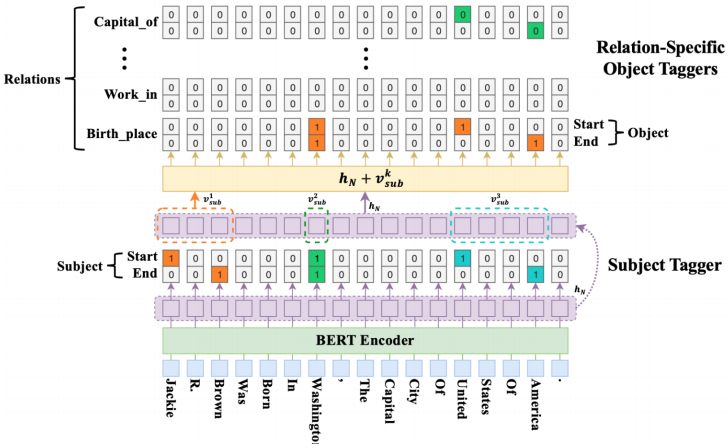


图 21: CasRel 模型框架

i. **BERT Encoder**: 对句子中的单词进行编码，本实验中使用的是谷歌发布的 “bert-base-chinese” 预训练的中文语言模型。
BERT（Bidirectional Encoder Representations from Transformers）是一种基于 Transformer 架构的预训练模型。它包含多个层，每个层都有多个自注意力头，用于捕捉文本中的双向上下文关系。Encoder 接收 token 序列作为输入，通过多头自注意力机制生成每个 token 的表示。这使得表示能够包含全局上下文信息，有助于提高模型在各种 NLP 任务中的性能。

ii. **Subject Tagger**: 识别出文本中的实体 S。
本部分的主要内容是识别出所有可能的主语实体 S。先对 Bert Encoder 获取到的词的隐层进行解码，构建二分类器预测 S 的 start 和 end 位置，对每一个词计算其作为 start 和 end 的概率，并根据一个阈值确定其为 1 还是 0(3)。
在 CasRel 论文中只需要找出实体边界即可，实体的开始和结束都使用 1 来表示。

$$\begin{aligned} p_i^{start_S} &= \sigma(W_{start} x_i + b_{start}) \\ p_i^{end_S} &= \sigma(W_{end} x_i + b_{end}) \end{aligned} \tag{3}$$

iii. **Relation-specific Object Taggers**: 根据实体 S，寻找可能的关系 R 和实体 O。在此层结构里，对于识别出来的每一个实体 S，遍历所有的关系 R，构建二分类器预测实体 O 的 start 和 end 位置，和 Subject Tagger 类似 (4)。

$$\begin{aligned} p_i^{start_O} &= \sigma(W_{start}^R (x_i + v_{sub}^k) + b_{start}^R) \\ p_i^{end_O} &= \sigma(W_{end}^R (x_i + v_{sub}^k) + b_{end}^R) \end{aligned} \tag{4}$$

2. **NER_IDCNN**

(a) 迭代扩张卷积 (ID-CNN)
传统的方法是使用基于规则或统计的模型，但这些模型需要大量的人工特征工程，而且泛化能力有限。近年来，基于神经网络的模型取得了很好的效果，但这些模型通常使用循环神经网络（如 LSTM）来编码文本，这样会导致计算效率低下，而且难以捕捉长距离的上下文信息。
迭代扩张卷积 (ID-CNN)，可以有效地捕捉长距离的上下文信息，而不增加计算复杂度，是本模型的重点方法。它的基本思想是使用多层的一维卷积层，每层都有不同的扩张率 (dilation rate)。扩张率是指卷积核中每个元素之间的间隔，它决定了卷积核的感受野 (receptive field) 的大小。例如，扩张率为 1 的卷积核的感受野是 3，扩张率为 2 的卷积核的感受野是 5，扩张率为 4 的卷积核的感受野是 9，以此类推。通过使用不同的扩张率，可以让卷积层捕捉不同范围的上下文信息，而不增加参数的数量或计算的复杂度。

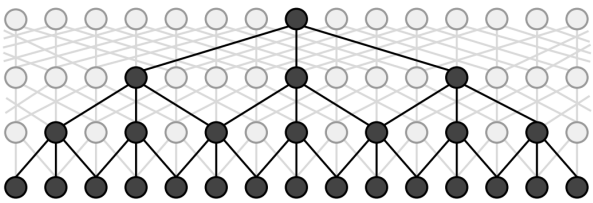


图 22: 感受野

迭代扩张卷积的公式如下：

假设输入的文本序列为 $x = (x_1, x_2, ..., x_n)$，其中 x_i 是第 i 个单词或字符的嵌入向量，n 是文本的长度。

假设第 l 层的卷积层有 m 个卷积核，每个卷积核的大小为 k，扩张率为 d。则第 l 层的输出为 $h^{(l)} = (h_1^{(l)}, h_2^{(l)}, ..., h_n^{(l)})$，其中 $h_i^{(l)}$ 是一个 m 维的向量，表示第 i 个位置的特征向量。$h_i^{(l)}$ 的计算公式为：

$$h_i^{(l)} = f(W^{(l)} * h_i^{(l-1)} + b^{(l)}) \tag{5}$$

其中 $W^{(l)}$ 是一个 $m \times m \times k$ 的张量，表示第 l 层的卷积核参数，$b^{(l)}$ 是一个 m 维的向量，表示第 l 层的偏置参数，f 是一个激活函数，例如 ReLU，$*$ 表示扩张卷积运算，定义为：

$$W^{(l)} * h_i^{(l-1)} = \sum_{j=1}^{k} W_j^{(l)} h_{i+(j-1)d}^{(l-1)} \tag{6}$$

其中 $W_j^{(l)}$ 是一个 $m \times m$ 的矩阵，表示第 l 层的第 j 个卷积核的切片，$h_{i+(j-1)d}^{(l-1)}$ 是一个 m 维的向量，表示第 $l-1$ 层的第 $i+(j-1)d$ 个位置的特征向量。如果 $i+(j-1)d$ 超出了文本的范围，那么 $h_{i+(j-1)d}^{(l-1)}$ 为零向量。

(b) 条件随机场 (CRF)

条件随机场 (CRF) 是一种概率图模型，用来考虑实体类别之间的转移约束，以提高模型的准确度。

CRF 对给定输入序列 X 条件下的输出标签序列 Y 建模，其条件概率分布表示为 $P(Y|X)$。CRF 中使用了一组特征函数，这些函数表示输入序列和输出标签之间的关系。每个特征函数对应于一个局部的观察和标签对。使用全局归一化来确保条件概率分布的合法性，以便于进行推断和学习。这涉及到对所有可能的标签序列的概率进行归一化。

在本模型中，CRF 层的参数为 A，是一个 $p \times p$ 的矩阵，表示不同类别之间的转移概率。CRF 层的目标函数为：

$$\mathcal{L}(x, y) = -\log \frac{\exp(\text{score}(x, y))}{\sum_{y'} \exp(\text{score}(x, y'))} \tag{7}$$

其中 $\text{score}(x, y)$ 表示给定输入序列 x 和输出序列 y 的得分，定义为：

$$\text{score}(x, y) = \sum_{i=1}^{n} y_i^T W^{(o)} h_i^{(L)} + b^{(o)} + \sum_{i=1}^{n-1} A_{y_i, y_{i+1}} \tag{8}$$

通过最小化这个负对数似然函数，我们实际上在训练模型时考虑了标签之间的依赖关系，以及每个位置上的标签预测。

(c) 核心代码解读

下面我们重点解读模型中的核心代码部分，以进一步阐述模型核心方法 IDCNN 的工作原理。

IDCNN layer:

```
def IDCNN_layer(self, model_inputs, name=None):
    """
    :param model_inputs: [batch_size, num_steps, emb_size]
    :return: [batch_size, num_steps, cnn_output_width]
    """
    model_inputs = tf.expand_dims(model_inputs, 1)
    reuse = False
    if not self.is_train:
        reuse = True
    with tf.variable_scope("idcnn" if not name else name):
        shape=[1, self.filter_width, self.embedding_dim,
                self.num_filter]
        print(shape)
        filter_weights = tf.get_variable(
            "idcnn_filter",
            shape=[1, self.filter_width, self.embedding_dim,
                 self.num_filter],
            initializer=self.initializer)

        layerInput = tf.nn.conv2d(model_inputs,
                            filter_weights,
                            strides=[1, 1, 1, 1],
                            padding="SAME",
                            name="init_layer")
        finalOutFromLayers = []
        totalWidthForLastDim = 0
        for j in range(self.repeat_times):
            for i in range(len(self.layers)):
                dilation = self.layers[i]['dilation']
                isLast = True if i == (len(self.layers) - 1) else False
                with tf.variable_scope("atrous-conv-layer-%d" % i,
                                       reuse=tf.AUTO_REUSE):
                    w = tf.get_variable(
                        "filterW",
                        shape=[1, self.filter_width, self.num_filter,
                               self.num_filter],
                        initializer=tf.contrib.layers.xavier_initializer())
                    b = tf.get_variable("filterB", shape=[self.num_filter])
                    conv = tf.nn.atrous_conv2d(layerInput,
```

```
                                    w,
                                    rate=dilation,
                                    padding="SAME")
                conv = tf.nn.bias_add(conv, b)
                conv = tf.nn.relu(conv)
                if isLast:
                    finalOutFromLayers.append(conv)
                    totalWidthForLastDim += self.num_filter
                layerInput = conv
        finalOut = tf.concat(axis=3, values=finalOutFromLayers)
        keepProb = 1.0 if reuse else 0.5
        finalOut = tf.nn.dropout(finalOut, keepProb)

        finalOut = tf.squeeze(finalOut, [1])
        finalOut = tf.reshape(finalOut, [-1, totalWidthForLastDim])
        self.cnn_output_width = totalWidthForLastDim
        return finalOut
```

解读如下:

- Input Transformation：model_inputs 被扩展一个维度，将其形状变为 [batch_size, 1, num_steps, emb_size]，以适应卷积操作。
- Filter Initialization：filter_weights 是卷积操作的权重，其形状为 [1,filter_width, embedding_dim, num_filter]。通过卷积核的初始化来学习输入数据的局部和全局特征。
- Initial Convolution Layer：执行第一层卷积操作（init_layer），将输入通过卷积核进行卷积。
- Iterated Dilated Convolution Layers：使用迭代扩张卷积（atrous convolution）进行多层卷积操作。迭代中的每一层都具有不同的扩张率（dilation），通过增加感受野，模型能够捕捉更长距离的依赖关系。并使用 ReLU 激活函数。
- Dropout：在最后一层卷积操作之后应用 Dropout 操作，以防止过拟合。
- Output Transformation：通过对输出进行挤压和重塑，得到最终的输出，形状为 [batch_size * num_steps, cnn_output_width]。

CRF Loss layer:

```
def loss_layer(self, project_logits, lengths, name=None):
    """
    calculate crf loss
    :param project_logits: [1, num_steps, num_tags]
    :return: scalar loss
    """
    with tf.variable_scope("crf_loss" if not name else name):
        small = -1000.0
        # pad logits for crf loss
        start_logits = tf.concat(
            [small * tf.ones(shape=[self.batch_size, 1, self.num_tags]),
                tf.zeros(shape=[self.batch_size, 1, 1])], axis=-1)
        pad_logits = tf.cast(small * tf.ones([self.batch_size,
            self.num_steps, 1]), tf.float32)
        logits = tf.concat([project_logits, pad_logits], axis=-1)
        logits = tf.concat([start_logits, logits], axis=1)
        targets = tf.concat(
            [tf.cast(self.num_tags*tf.ones([self.batch_size, 1]), tf.int32),
                self.targets], axis=-1)

        self.trans = tf.get_variable(
            "transitions",
            shape=[self.num_tags + 1, self.num_tags + 1],
            initializer=self.initializer)
        log_likelihood, self.trans = crf_log_likelihood(
            inputs=logits,
            tag_indices=targets,
            transition_params=self.trans,
            sequence_lengths=lengths+1)
        return tf.reduce_mean(-log_likelihood)
```

解读如下:

- Logits Padding:通过在 logits 前面和末尾添加 padding,以处理序列两端的标签。start_logits 在序列起始处添加 padding，pad_logits 在序列末尾添加 padding。
- CRF Transition Parameters：self.trans 是 CRF 层的转移参数矩阵，形状为 [num_tags + 1, num_tags + 1]。num_tags + 1 表示在标签集合中添加了一个额外的"start" 标签。
- CRF Log-Likelihood Calculation：调用 crf_log_likelihood 函数计算 CRF 的对数似然。inputs 是经过 padding 处理的 logits，tag_indices 是目标标签序列，transition_params 是转移参数，sequence_lengths 是真实序列的长度。返回的 log_likelihood 包含了每个序列的对数似然值。
- Loss Calculation：计算负对数似然的均值作为最终的损失值。负对数似然越小，表示模

型预测的序列越接近真实序列，因此损失越小。

3. **RE_BGRU_2ATT**

(a) 双向门控循环单元 (BGRU)

本模型是在 LSTM 的基础上加以改进，使用 GRU 模型。GRU 是一种循环神经网络（RNN）的变体，旨在简化 LSTM 的结构并减少参数。它通过使用门控机制来选择性地更新和传递信息，从而减轻了梯度消失问题。GRU 中的门控单元决定了哪些信息应该被保留和传递给下一时间步。

BGRU（Bidirectional Gated Recurrent Unit）是一种双向门控循环单元模型，用于处理序列数据。它是 GRU（Gated Recurrent Unit，门控循环单元）的一种变体，添加了一个反向的 GRU 层。这使得模型能够同时考虑序列数据的过去和未来上下文信息。BGRU 的前向和后向层分别处理输入序列的正向和逆向顺序，并将它们的输出进行连接或合并。

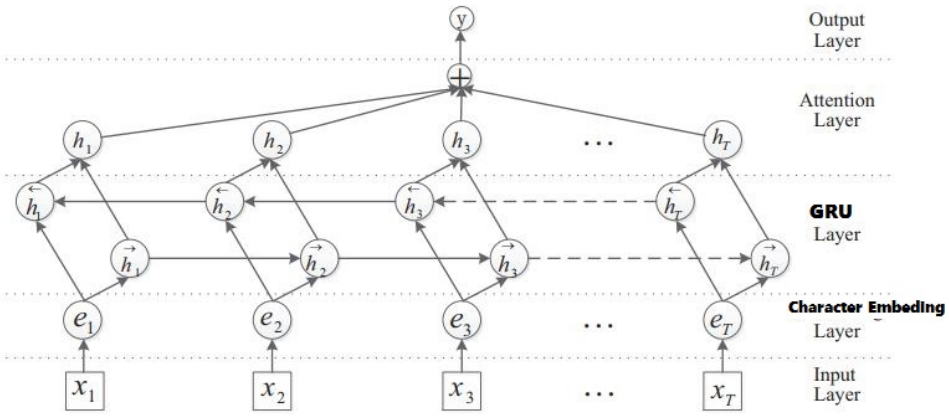


图 23: BGRU

我们以前向 GRU 的计算过程为例，分析 BGRU 的公式。

首先更新门决定了前一时刻隐藏状态 h_{t-1} 和当前输入 x_t 对当前时刻的隐藏状态 h_t 有多大的影响。通过应用 Sigmoid 函数，它输出一个介于 0 和 1 之间的值，表示保留或丢弃前一时刻的隐藏状态的程度。

$$z_t = sigmoid(W_z * x_t + U_z * h_{t-1} + b_z) \tag{9}$$

重置门决定了前一时刻隐藏状态 h_{t-1} 对当前时刻的输入 x_t 的影响。它通过应用 Sigmoid 函数，决定了在计算当前时刻的候选隐藏状态 $h_t^{'}$ 时，应该丢弃多少来自前一时刻隐藏状态的信息。

$$r_t = sigmoid(W_r * x_t + U_r * h_{t-1} + b_r) \tag{10}$$

候选隐藏状态是根据当前输入 x_t 和重置门的信息计算得到的。它结合了当前输入和之前时刻的隐藏状态，以及重置门确定的要保留的前一时刻隐藏状态的部分信息。通过应用双曲正切激活函数，它将值映射到 [-1, 1] 的范围内。

$$h_t^{'} = tanh(W_h * x_t + U_h * (r_t\ h_{t-1}) + b_h) \tag{11}$$

当前隐藏状态是根据更新门、前一时刻隐藏状态和候选隐藏状态计算得到的。它考虑了前一时刻的隐藏状态和当前时刻的候选隐藏状态，通过加权求和的方式得到当前时刻的隐藏状态。更新门决定了前一时刻隐藏状态和候选隐藏状态的权重。

$$h_t = (1 - z_t)\, h_{t-1} + z_t\, h_t^{'} \tag{12}$$

后向 GRU 的公式与前向 GRU 类似，只是时间步的方向相反，因此不做重复阐述。通过前向 GRU 和后向 GRU 的计算，BGRU 模型能够同时考虑过去和未来的上下文信息，从而更好地捕捉序列数据中的特征和依赖关系。

(b) 两层注意力机制 (2ATT)

注意力机制（Attention Mechanism）是一种在机器学习和自然语言处理领域广泛应用的技术，用于在序列数据中对不同位置或特征的重要性进行加权，以便模型能够更有针对性地关注和利用关键信息。

在本实验中，我们所运用的模型是在在双向 GRU 的基础上，引入了两层注意力机制，分别是双向 GRU 加字级别注意力机制，和句子级别的注意力机制，如下图所示。

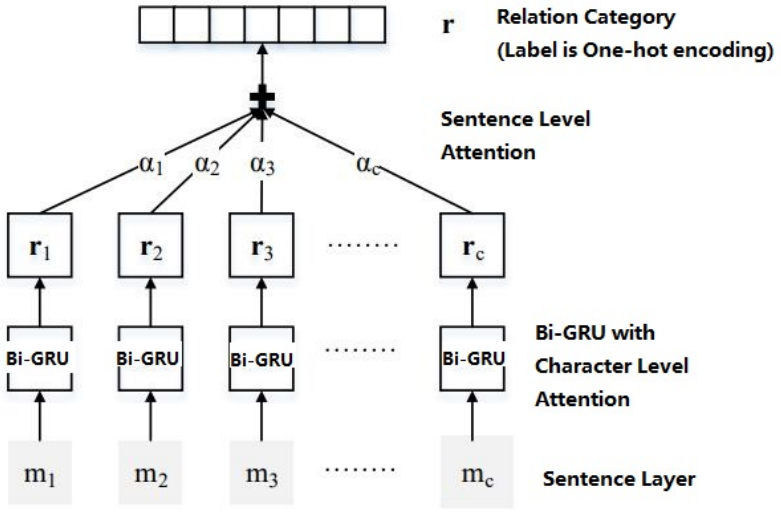


图 24: 两层注意力机制

- 双向 GRU 加字级别注意力机制 (Bi-GRU with Character Level Attention) 是一种结合了双向 GRU 和字级别注意力机制的模型。首先，通过字符嵌入将输入文本序列中的每个字符转换为连续向量表示。然后，使用双向 GRU 对字符级别的表示进行正向和反向的处理，捕捉到输入序列前后位置的语义信息。接下来，应用字级别注意力机制，计算每个字符表示与注意力权重的加权和，以获取关键字符的重要性。最后，将双向 GRU 的输出与字级别注意力的加权和结合，生成整个文本序列的上下文表示，提高对文本序列的表示能力。
- 句子级别注意力机制 (Sentence Level Attention) 是一种用于文本处理的注意力机制，它能够在文本序列中捕捉句子级别的语义信息。首先，将输入的文本序列经过编码，例如使用循环神经网络（RNN）或卷积神经网络（CNN）将每个单词或字符转换为向量表示。然后，根据编码后的文本表示计算每个位置的注意力权重，以确定句子级别的关注

程度。注意力权重可以通过不同的计算方法得到，如点积注意力、加性注意力或多层感知机（MLP）注意力。最后，根据注意力权重对编码后的文本表示进行加权求和操作，得到句子级别的表示，该表示捕捉了输入文本序列中重要的句子级别语义信息。

(c) 核心代码解读

Bi-GRU layer:

```
with tf.variable_scope('GRU_FORWARD') as scope:
         for step in range(num_steps):
            if step > 0:
               scope.reuse_variables()
            (cell_output_forward, state_forward) =
                cell_forward(inputs_forward[:, step, :], state_forward)
            outputs_forward.append(cell_output_forward)

      outputs_backward = []

      state_backward = self._initial_state_backward
      with tf.variable_scope('GRU_BACKWARD') as scope:
         for step in range(num_steps):
            if step > 0:
               scope.reuse_variables()
            (cell_output_backward, state_backward) =
            cell_backward(inputs_backward[:, step, :], state_backward)
            outputs_backward.append(cell_output_backward)

      output_forward = tf.reshape(tf.concat(axis=1,
          values=outputs_forward), [total_num, num_steps, gru_size])
      output_backward = tf.reverse(
         tf.reshape(tf.concat(axis=1, values=outputs_backward),
             [total_num, num_steps, gru_size]),[1])
```

- 首先定义了两个变量作用域，分别对应正向和反向的 GRU。
- 使用循环来迭代处理输入的每个时间步。在正向的 GRU 中，通过调用 cell_forward 和 state_forward 来获取每个时间步的输出和状态。这些输出被添加到 outputs_forward 列表中。
- 在反向的 GRU 中，使用类似的循环来获取反向的输出和状态，并将它们添加到 outputs_backward 列表中。
- 通过 tf.concat 函数将正向和反向的输出连接起来，然后使用 tf.reshape 函数对形状进行调整，得到正向和反向输出的张量。

character-level attention layer:

```
output_h = tf.add(output_forward, output_backward)
attention_r = tf.reshape(tf.matmul(tf.reshape(tf.nn.softmax(tf.reshape
(tf.matmul(tf.reshape(tf.tanh(output_h), [total_num * num_steps,
    gru_size]),attention_w),
[total_num, num_steps])), [total_num, 1, num_steps]), output_h),
    [total_num, gru_size])
```

- 将正向和反向的输出相加，得到 output_h，它是一个形状为 [total_num, num_steps, gru_size] 的张量。
- 使用 tf.tanh 函数对 output_h 进行激活，将其形状调整为 [total_num * num_steps, gru_size]，并与 attention_w 进行矩阵乘法运算。
- 通过 tf.nn.softmax 函数对结果进行归一化，得到一个形状为 [total_num, 1, num_steps] 的张量，表示每个字的注意力权重。
- 将注意力权重与 output_h 进行矩阵乘法运算，得到形状为 [total_num, gru_size] 的 attention_r 张量。
- 通过 tf.reshape 函数将 attention_r 的形状调整为 [total_num, gru_size]，得到字级别的注意力表示

sentence-level attention layer:

```
for i in range(big_num):

   sen_repre.append(tf.tanh
   (attention_r[self.total_shape[i]:self.total_shape[i + 1]]))
   batch_size = self.total_shape[i + 1] - self.total_shape[i]

   sen_alpha.append(tf.reshape(tf.nn.softmax(tf.reshape(tf.matmul
   (tf.multiply(sen_repre[i], sen_a), sen_r), [batch_size])),[1,
       batch_size]))

   sen_s.append(tf.reshape(tf.matmul(sen_alpha[i], sen_repre[i]),
       [gru_size, 1]))
   sen_out.append(tf.add(tf.reshape(tf.matmul(relation_embedding,
       sen_s[i]), [self.num_classes]), sen_d))

   self.prob.append(tf.nn.softmax(sen_out[i]))

   with tf.name_scope("output"):
      self.predictions.append(tf.argmax(self.prob[i], 0,
          name="predictions"))

   with tf.name_scope("loss"):
```

```
    self.loss.append(tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits
    (logits=sen_out[i], labels=self.input_y[i])))
        if i == 0:
            self.total_loss = self.loss[i]
        else:
            self.total_loss += self.loss[i]

    # tf.summary.scalar('loss',self.total_loss)
    # tf.scalar_summary(['loss'],[self.total_loss])
    with tf.name_scope("accuracy"):
        self.accuracy.append(
            tf.reduce_mean(tf.cast(tf.equal(self.predictions[i],
                tf.argmax(self.input_y[i], 0)), "float"),
                       name="accuracy"))

# tf.summary.scalar('loss',self.total_loss)
tf.summary.scalar('loss', self.total_loss)
# regularization
self.l2_loss = tf.contrib.layers.apply_regularization
(regularizer=tf.contrib.layers.l2_regularizer(0.0001),
weights_list=tf.trainable_variables())
self.final_loss = self.total_loss + self.l2_loss
tf.summary.scalar('l2_loss', self.l2_loss)
tf.summary.scalar('final_loss', self.final_loss)
```

- 首先使用双向 GRU 对输入文本进行编码，得到正向和反向的输出。
- 通过词级别的注意力层，对每个词的表示进行加权组合，得到句子级别的表示。
- 通过将句子级别的表示与关系嵌入向量相乘并添加偏置项，得到每个类别的预测分数。
- 模型使用交叉熵损失函数进行训练，并计算准确率作为评估指标。
- 应用 L2 正则化来控制模型的复杂度，并将 L2 正则化损失添加到总体损失中。

4. **PL-Marker(Packed Levitated Markers)**

(a) 概述

跨度表示（span representation）近年来为命名实体识别，关系和时间提取，核心参照解析，语义角色标注等各种 NLP 应用提供了不少便利，此模型就是通过引入打包悬浮标记（Packed Levitated Markers）——通过在编码器中战略性的打包标记来考虑跨度（对）之间的的相互关系，实现跨度表示对 NLP 的应用，它设计了一种面向主体的打包策略，该策略打包每个主体及所有对象，以模拟同一主体跨度对之间的相互关系。在六个 NER 邻域都取得了进步，速度超过了以前的先进模型。

(b) 模型思路的基础概览

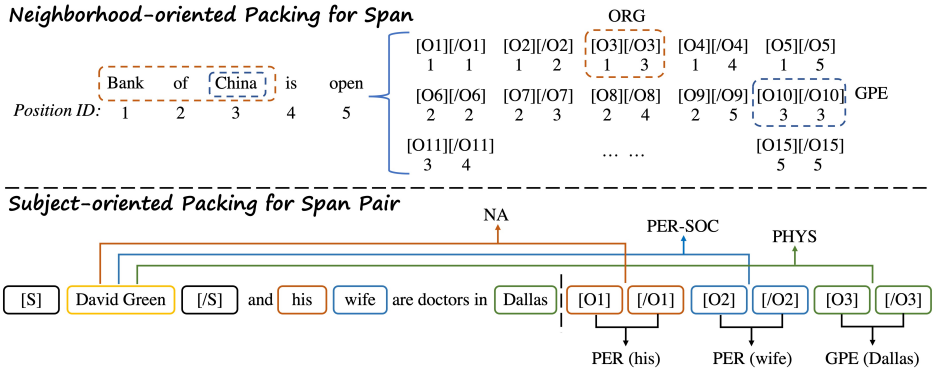


图 25: 面向邻域的打包策略和面向主题的打包策略概览

如图所示，首先以起始标记的位置作为第一个关键字，以结束标记的位置作为第二个关键字，对悬浮标记进行排序。然后，将它们分成大小不超过 K 的组，并将相邻的跨度集合到同一组中。对每组标记进行打包，并在多次运行中对其进行分散处理。形式上，给定 N 个文本标记序列 X= x1 ,... . 1 和最大跨度长度 L，我们将候选跨度集定义为 S(X) =(1, 1), ..,(1, L), ...,(N, N L), ..,(N, N)). 首先将 S(X) 依次分成大小不超过 K 的多个组。悬浮标记被用作实体标记的近似值，它允许模型同时对多对实体进行分类，以加快为 S 中的每个跨度关联一对浮动标记。然后，我们将文本标记和插入的浮动标推理过程。A 与跨度相关的一对悬浮标记由起始标记和结束标记组成。这两个标记共享相同的位置的起始和结束标记嵌入。例如，我们将悬浮标记应用于一个典型的重叠跨度分类任务–NER，其目的是指定实体类型或非实体类型。T-Concat 可以为句子中每个可能的跨度提供实体类型。我们通过打包的悬浮标记从 PLM 中获得跨度表示，然后结合 PL-Marker 和 T-Concat 的功能，更好地预测候选跨度的实体类型。

(c) 模型结构

PL-Marker 使用的模型就是 PLM，与 PURE-Approx 类似：

- 拼接在句子后面的悬浮标记对中的 start marker 与其在句子中对应的 span 的 start token 共享 position embedding，end marker 与对应 span 的 end token 共享 position embedding；
- 通过 attention mask 矩阵进行限制，使得悬浮标记可以看到它的搭档标记和前面的文本，看不到其他悬浮标记对，文本仅可以看到文本

(d) NER 部分：Neighborhood-oriented Packing for span

这部分采用的悬浮标记，将所有的可能的实体 span 的悬浮标记对都放在句子最后面。但是这样就出现了一个问题，因为要遍历句子中所有可能的 span，而 PLM 能处理的句子长度有限。因此他们提出了 Packing 的策略，在 Packing 的时候，考虑到为了更好的分清楚 span 的边界（更重要的是区分同一个词为开头的 span 的差别），会将 span 相近的放在一起，就是将以开头相同的或者相近的放在一个样本中。对于一个 token 数量为 N 的句子，规定最大的 span 长度为 L，具体步骤如下：

- 首先，对所有的悬浮标记对 (一个开始标记，一个结束标记) 进行排序，排序的方式是按照它们每一对悬浮标记所代表的的 span 的 start token 的位置，以及 end token 的位置，从小到大进行排序，得到排序后的候选 span 列表；

- 然后，将所有的悬浮标记拆分成 K 个组，这样的话相邻的 span 的悬浮标记就会被分在一个组里，然后每组拼接分别拼接在句子后面，作为 K 个样本
- 最后，将样本送进 PLM，对于每一对悬浮标记对 s_i=(a,b)，分别将他们的开始标记的表征 h_a^{start} 和结束标记的表征 h_b^{end} 拼接在一起，作为其对应 span 的表征:

$$\psi(s_i) = [h_a^{start}; h_b^{end}] \tag{13}$$

- 而在进行 NER 的时候，他们是通过上面的步骤获取到 span 表征（也就是 PL-Marker 抽取到的 span 特征），然后与 T-Concat 方法抽取的 span 表征合并起来去预测 entity type。

(e) RE 部分：Subject-oriented Packing for span pair
这部分采用固定标记和悬浮标记混用的方式，原因是如果也采用 PURE-Approx 的那种 subject span levitated marker 与 object span levitated marker 都拼接在后面的情况的话，虽然可以用 attention mask 去把两对 marker 绑定在一起，但是这样的话，那么它就不太能识别出它同 span 的另一个 marker。具体做法如下：

- 对于一个句子，以及其中的 subject span 和它对应的 object spans，构成一条训练样本，其中 subject span 采用固定标记，也就是在句子中 span 单词的前后直接插入 [S] 和 [/S] 两个标记，然后将它对应的候选 Object span 用悬浮标记的方式拼接在文本后面。
- 然后把样本送进 Pretrained Encoder，对于样本中的每一个 span 对 $s_{subject}$ =(a,b) 和 s_{object} =(c,d) ，将 Subject span 前后的固定标记的表征 h_{a-1} 和 h_{b+1} 以及一对 object span 的悬浮标记的表征 h_c^{start} 和 h_d^{end} 拼接在一起，作为这一对 span pair 的表征：

$$\phi(s_{subject}, s_{object}) = [h_{a-1}; h_{b+1}; h_c^{start}; h_d^{end}] \tag{14}$$

- 为了建模实体类型与关系类型之间的关系，他们还增加了预测 object entity 的辅助 loss
- 为了增加一些补充信息，他们新增了从 object 到 subject 的反向关系的预测，从而实现了双向关系的预测，其实就是实现了一个 Object-oriented packing strategy

2.2.5 网页展示

由于本次大作业分别用多个模型实现了命名实体识别 (NER) 和关系抽取 (RE) 的任务，综合比较模型效果以及时间精力等多种因素，这里我们选择基于 CasRel 模型来进行最终测试结果的展示。

app.py 文件实现了通过网页来展示最终的测试结果，可视化前后端数据的思路大致如下：采用 Flask 框架，配合 Ajax 方法，使用 json 文件实现前后端口数据的传输，基于 D3.js 实现话题-观点知识图谱的生成。

将测试模型的函数规范为一种方法，配合使用 Flask 框架如下：

```
@app.route('/index')

def index():
    return render_template('index.html')
```

当网页 Ajax 访问一个地址。其访问过程类似于浏览器的地址栏，将回到后台读取代码中，匹配的 url(也就是 app.route())，将会进入这个方法。

数据交互的过程与 servlet 类似，使用 post、get 请求，其 url 对应，这里的 app.route(自定义的方法名字)，其端口下的方法。

此时，前端提交 post 请求，请求访问 Python 项目的方法 (即本函数中的测试过程)。后台 python 逻辑运行操作得出的结果将存在 json 文件中，用来返回给页面。

```
def getInfo():
   test_data = request.values.get("testdata")
...
数据处理 得到result
...

   return json.dumps(result, ensure_ascii=False)
```

前端页面写 Ajax，发送请求，成功时会接收到后台输入了数据的 json 文件，依照文件格式，对数据再进行相关处理后，展示到 html 页面上。

```
function toGetData(){
         var val = $("#t6").text(); //文本数据框的内容
         //去后台调用模型做逻辑操作
            $.ajax({
                     url:'/api/getInfo',
                     data:{"testdata":val},
                     success:function(data){
                        data = data.replace("\"","").replace("\"","");
                        var back = data;
                        var allhtml = {};
                        var allhtml2 = [];
...
                        $("#t1").html(allhtml);
                        $("#t2").html(allhtml2);
                     },
                  })
         //将文本传给节点数据和关系数据框的内容
      }
```

由此通过 json 文件，实现前后端数据同步传输的过程。

接着，基于 D3.js 实现通过传输的数据绘制关系图谱。

```
   // 初始化图谱
function initKG(data, config, container) {
   var nodeDict = data.nodes; // 节点信息
   var links = data.links; // 关系信息
   var nodes = {};      // 节点字典

```

```
// 将链接中的source和target名称与节点确认，并存储在nodes字典中
links.forEach(function (link) {
   link.source = nodeDict[link.source];
   nodes[link.source.name] = link.source;
   link.target = nodeDict[link.target];
   nodes[link.target.name] = link.target;
});

// 默认的节点配色方案
var defaultNodeColor = [
   { fill: "rgb(249, 235, 249)", stroke: "rgb(162, 84, 162)", text:
       "rgb(162, 84, 162)" },
   { fill: "#ccc", stroke: "rgb(145, 138, 138)", text: "#333" },
   // ... 其他颜色方案
];

// 默认的关系配色方案
var defaultLinkColor = [
   { color: "rgb(162, 84, 162)" },
   // ... 其他颜色方案
];

// 为节点分配颜色方案
var colorDict = new Array();
var point = 0;
Object.keys(data.nodes).forEach(function (key) {
   var type = data.nodes[key].type == null ? (data.nodes[key].type =
       "default") : data.nodes[key].type
   if (colorDict[type] == null) {
      colorDict[type] = defaultNodeColor[point]
      point = (point + 1) % defaultNodeColor.length
   }
});

// 为链接分配颜色方案
var colorLinkDict = new Array();
point = 0;
Object.keys(data.links).forEach(function (key) {
   var type = data.links[key].type == null ? (data.links[key].type =
       "default") : data.links[key].type
   if (colorLinkDict[type] == null) {
      colorLinkDict[type] = defaultLinkColor[point]
      point = (point + 1) % defaultLinkColor.length
```

```
    }
});

var width = config.width ? config.width : 1560;
var height = config.height ? config.height : 800;

// 创建力导向图布局
var force = d3.layout.force()
    .nodes(d3.values(nodes))
    .links(links)
    .size([width, height])
    .linkDistance(120)
    .charge(-1200)
    .on("tick", tick)
    .start();

// 创建缩放功能
var zoom = d3.behavior.zoom()
    .scaleExtent([0, 2])
    .on("zoom", zoomed);

function zoomed() {
    svg.selectAll("g").attr("transform",
        "translate(" + d3.event.translate + ")scale(" + d3.event.scale + ")");
}

// 添加SVG元素
var svg = d3.select(container).append("svg")
    .attr("width", width)
    .attr("height", height)
    .call(zoom);

// 添加箭头
var marker = svg.append("marker")
    // ... 箭头样式配置

// 设置连接线
var edges_line = svg.append("g").selectAll(".edgepath")
    .data(force.links())
    .enter()
    .append("path")
    // ... 连线样式配置

```

```
    // 连线上的文字
    var edges_text = svg.append("g").selectAll(".edgelabel")
        .data(force.links())
        .enter()
        .append("text")
        // ... 文字样式配置

    // ... 其他代码

    function tick() {
        // ... 刷新页面函数的实现
    }

    function transform1(d) {
        return "translate(" + d.x + "," + d.y + ")";
    }
}
```

由此，通过 D3.js 库，我们实现了话题-观点图谱的生成。
此外，通过 html、css 完成网页界面的美化此处不做赘述，具体实现结果见下图。



图 26: 网页设计展示

2.3 评估方案

在评估方案中，我们分别就 Casrel、NER_IDCNN 和 RE_BGRU_2AT 模型训练过程中的 loss、accuracy、precision、f1、recall 指标进行评估，并以图像或表格的样式展示。

2.3.1 Casrel

下图是 CasRel 的 training loss 展示，共运行了 50 个 epoch，loss 由 22.8642 下降到 0.0646。

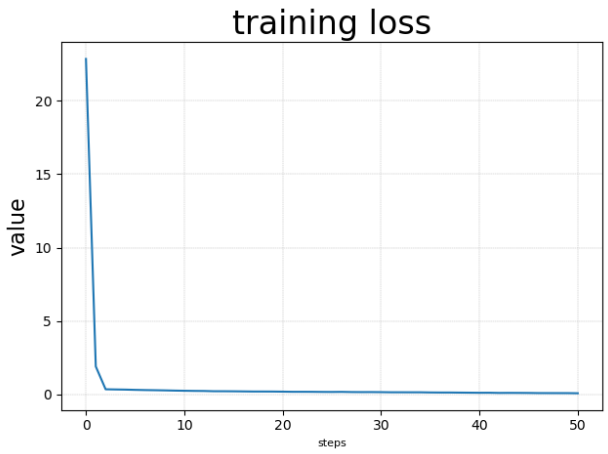


图 27: CasRel 的 training loss

CasRel 在测试集上的 precision、recall、Fβ1 的结果展示见下表:

Precision	Recall	Fβ1
76.83%	51.57%	61.72

可以看到 CasRel 在关系抽取任务中有不错表现，能够抽取到大量正确的有效关系。

2.3.2 NER_IDCNN

下图是 NER_IDCNN 的 training loss 图展示:

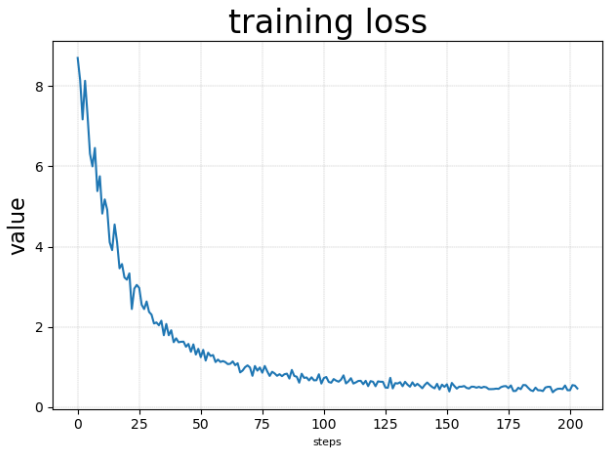


图 28: NER_IDCNN 的 training loss

下面是 NER_IDCNN 的 training accuracy 图展示:

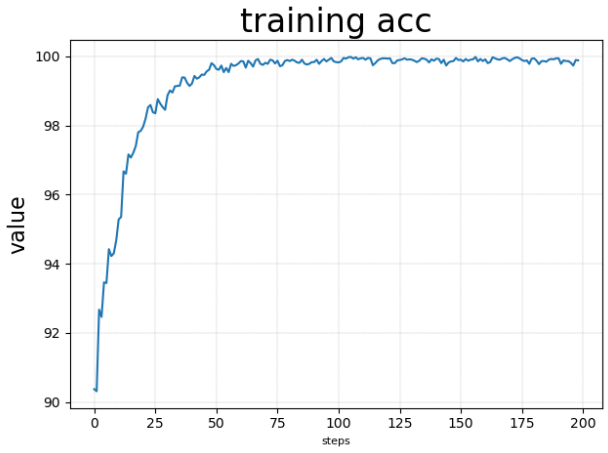


图 29: NER_IDCNN 的 training accuracy

对于 NER_IDCNN 的 best accuracy、precision、recall、Fβ1 的结果展示见下表:

Accuracy	Precision	Recall	Fβ1
99.89%	99.36%	99.57%	99.47

上述结果表明模型在整体上对样本进行了高度准确的分类，对于真实命名实体的识别性能，在处理不同类别样本上的综合性能都达到了较高水平。

2.3.3 RE_BGRU_2ATT

下图是 RE_BGRU_2ATT 的 training loss 图展示:

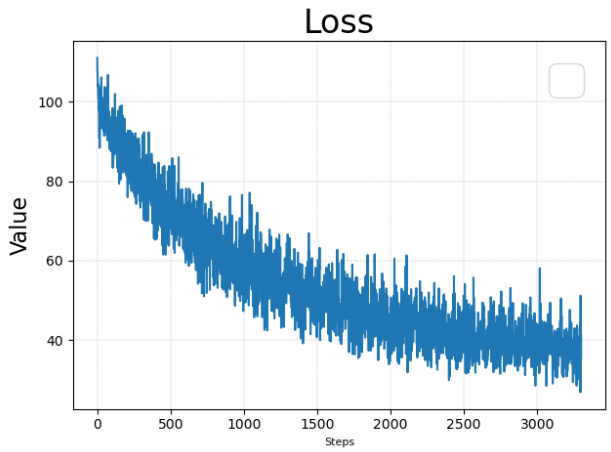


图 30: RE_BGRU_2ATT 的 training loss

下图是 RE_BGRU_2ATT 的 training average loss 图展示:

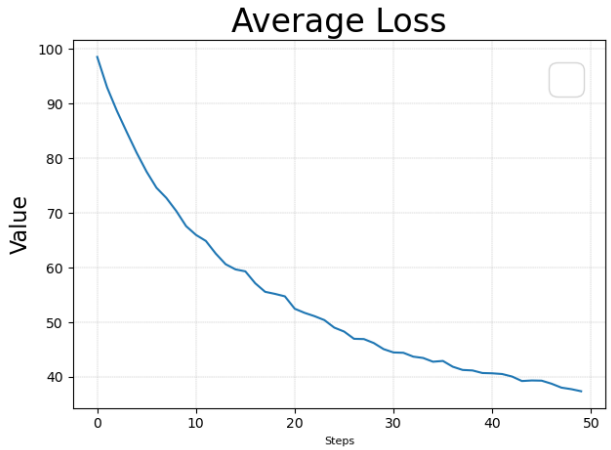


图 31: RE_BGRU_2ATT 的 training average loss

下面是 RE_BGRU_2ATT 的 training average accuracy 图展示:

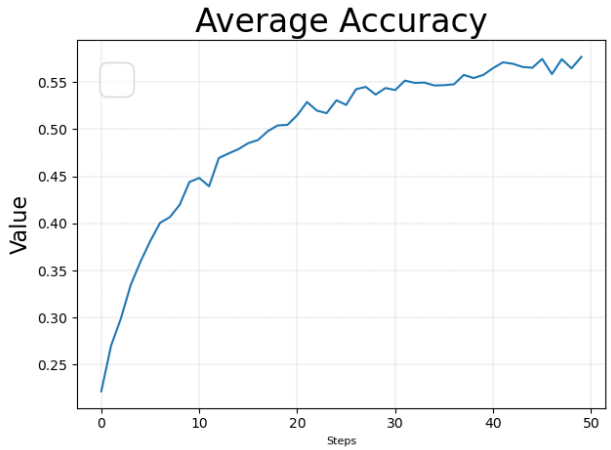


图 32: RE_BGRU_2ATT 的 training average accuracy

对于 RE_BGRU_2ATT 的 best accuracy 的结果展示见下表:

Best Accuracy
57.90%

2.4 实验成果

1. 使用 CasRel 模型实现测试集上的关系抽取任务
   “triple_list_gold”代表文本中标注的关系，“triple_list_pred”代表模型预测出的关系，“new”代表模型预测出的新关系，“lack”代表模型未预测出的关系，展示如下:


```
{
  "text": "中国 消费者 担忧 被 中美 贸易战 殃及  作为 对 特朗普 关税 的 反击 ， 中国 向 美国 大豆 等 商品 征收 报复性 关税 。
      官方 媒体 指责 美国 “ 贸易 霸凌 主义 ” ， 消费者 则 对 贸易战 可能 导致 的 进口商品 价格上涨 表示 担忧 。",
  "triple_list_gold": [
    {
      "subject": "美国",
      "relation": "-",
      "object": "媒体"
    },
    {
      "subject": "中国",
      "relation": "国家间",
      "object": "美国"
    },
    {
      "subject": "美国",
      "relation": "国家-机构",
      "object": "媒体"
    },
    {
      "subject": "中国",
      "relation": "国家-人物",
      "object": "特朗普"
    },
    {
      "subject": "中国",
      "relation": "国家-人物",
      "object": "消费者"
    },
    {
      "subject": "中国",
      "relation": "-",
      "object": "美国"
    }
  ],
  "triple_list_pred": [
    {
      "subject": "中国",
      "relation": "-",
      "object": "媒体"
    },
    {
      "subject": "中国",
      "relation": "国家-人物",
      "object": "特朗普"
    },
    {
      "subject": "中国",
      "relation": "-",
      "object": "美国"
    },
    {
      "subject": "中国",
      "relation": "国家间",
      "object": "美国"
    }
  ],
  "new": [
    {
      "subject": "中国",
      "relation": "-",
      "object": "媒体"
    }
  ],
  "lack": [
    {
      "subject": "美国",
      "relation": "-",
      "object": "媒体"
    },
    {
      "subject": "美国",
      "relation": "国家-机构",
      "object": "媒体"
    },
    {
      "subject": "中国",
      "relation": "国家-人物",
      "object": "消费者"
    }
  ]
}
```


图 33: CasRel 结果展示

2. 使用 NER_IDCNN 模型实现特定数据集上的命名实体识别任务，展示如下:


```
请输入测试句子:新西兰是唯一不是在中美之间选边站的五眼联盟最新新西兰外交部长马胡塔表示新西兰与中国的关系是必要的是我们最重要的外交关系之一自2017年以来中国一直是新西兰最大的贸易伙伴马胡塔我们对盟友要求同声批评中国感到极度不适
{'string': '新西兰是唯一不是在中美之间选边站的五眼联盟最新新西兰外交部长马胡塔表示新西兰与中国的关系是必要的是我们最重要的外交关系之一自2017年以来中国一直是新西兰最大的贸易伙伴马胡塔我们对盟友要求同声批评中国感到极度不适', 'entities': [{'word': '新西兰', 'start': 0, 'end': 3, 'type': '国家'}, {'word': '中', 'start': 9, 'end': 10, 'type': '国家'}, {'word': '美', 'start': 10, 'end': 11, 'type': '国家'}, {'word': '五眼联盟', 'start': 17, 'end': 21, 'type': '机构'}, {'word': '新西兰', 'start': 23, 'end': 26, 'type': '国家'}, {'word': '马胡塔', 'start': 30, 'end': 33, 'type': '人物'}, {'word': '新西兰', 'start': 35, 'end': 38, 'type': '国家'}, {'word': '中国', 'start': 39, 'end': 41, 'type': '国家'}, {'word': '中国', 'start': 69, 'end': 71, 'type': '国家'}, {'word': '新西兰', 'start': 74, 'end': 77, 'type': '国家'}, {'word': '马胡塔', 'start': 84, 'end': 87, 'type': '人物'}, {'word': '中国', 'start': 98, 'end': 100, 'type': '国家'}]}
```


图 34: NER_IDCNN 结果展示

3. 使用 RE_BGRU_2ATT 模型实现特定数据集上的关系抽取任务，展示如下:

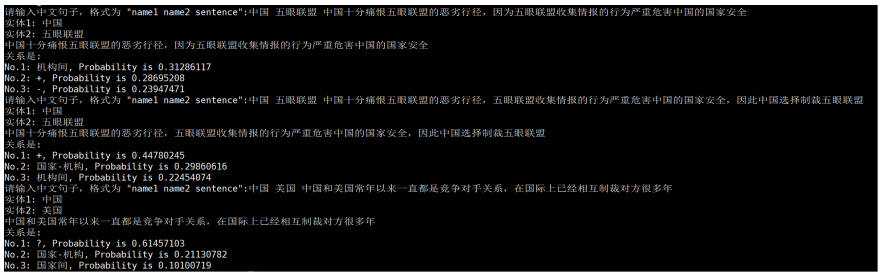


图 35: RE_BGRU_2ATT 结果展示

4. 我们将实验项目开源在 github 上,以供舆情分析相关工作者学习参考,链接如下:mumuyeye/Public-Opinion-Analysis: 武汉大学 2021 级网安舆情分析大作业 (github.com)

2.5 实验结果分析

1). 实体 & 关系抽取

这里我们使用测试集训练，点击实体 & 关系抽取按键后，模型的输出结果与人工标注结果大致一致，具体结果如下图所示。

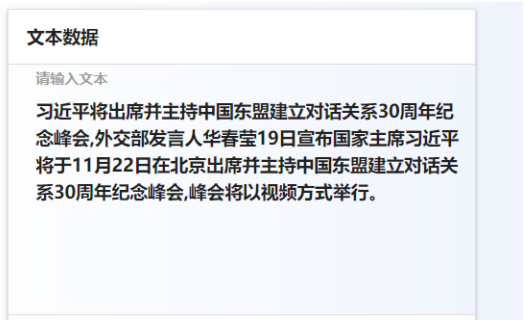


图 36: 文本数据



图 37: 实体抽取结果



图 38: 关系抽取结果

通过上面的图，我们可以看到，实体抽取以及关系抽取任务，模型完成的效果都比较不错，与人工标注的结果大致相同。

值得一提的是，这里为了方便后续关系图谱的生成，对原始输出数据进行了相关处理，原始输出数据格式下图所示。

```
Total params: 101,690,898
Trainable params: 101,690,898
Non-trainable params: 0
_____________________________________________________________________________________________
1it [00:03,  3.50s/it]
|<外交部，华春莹，机构-人物>|<中国，东盟，国家-机构>|<中国，习近平，国家-人物>|<中国，东盟，+>
```

图 39: 原始处理数据

2). 话题-观点图谱生成

点击话题-观点图谱生成按键后，得到的图谱结果如下图所示。

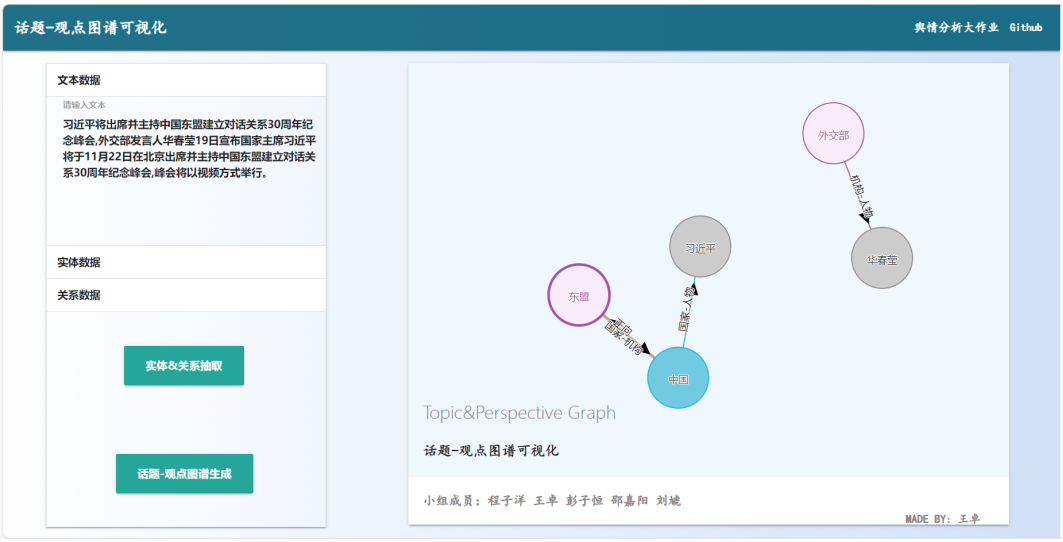


图 40: 话题-观点图谱

通过观察话题-观点图谱，我们成功将“习近平将出席并主持中国东盟建立对话关系 30 周年纪念峰会。外交部发言人华春莹 19 日宣布，国家主席习近平将于 11 月 22 日在北京出席并主持中国东盟建立对话关系 30 周年纪念峰会。峰会将以视频方式举行。”这句文本中的有关实体 (国家、机构、人物) 提取出来，并提取各实体之间的关系，其中包含不同实体之间的正向、负向、中立观点，以图谱的方式展示。至此成功完成了话题-观点图谱搭建任务。

2.6 创新性说明

2.6.1 CasRel

CasRel 采用了参数共享的机制，这意味着模型通过共享一部分参数来学习实体和关系之间的关联。这种方法的优势在于提高了模型的泛化能力，使得模型能够更好地适应不同类型和语境的文本。通过共享参数，模型可以更有效地学习抽取实体和关系的特征，从而在关系抽取任务中表现更为优越。

CasRel 的创新之处在于对子层的深入设计。这可能包括对实体和关系表示学习的改进，以及更复杂的特征提取和关系分类策略。子层的设计影响着模型对文本信息的理解和抽取关系的能力。通过精心设计的子层，CasRel 能够更好地捕捉实体之间的复杂关系，从而提高了关系抽取的准确性和鲁棒性。

CasRel 关注于关系抽取任务中的三元组重叠问题。这一问题意味着在同一文本中可能存在多个实体之间的多个关系，而模型需要能够有效地区分和抽取这些关系。通过特别关注这一问题，CasRel 提出了一种新的角度来解析关系抽取任务，使其不再局限于传统的多分类任务，而更加注重实体之间复杂关系的建模。让关系抽取任务从传统的多分类任务中跳脱出来。这意味着模型不仅仅将关系抽取视为简单的分类问题，而更注重于处理实体之间的语义关联。这一改进大幅度提高了关系抽取任务的各项指标，使得模型更适用于处理真实世界中的复杂文本情境。

2.6.2 NER_IDCNN

NER_IDCNN 使用新颖的迭代扩张卷积技术是一项创新，它在不增加计算复杂度的情况下，有效地捕捉了长距离的上下文信息。这一技术的关键在于扩张卷积，它允许卷积核的感受野随着层数的增加而呈指数级增长，而无需增加卷积核的大小或参数的数量。这一设计使得模型能够更好地理解文本的语义和结构，从而显著提高了命名实体识别任务的性能。

传统的卷积操作受限于固定的感受野，可能难以捕捉长距离的依赖关系。而迭代扩张卷积通过迭代地应用扩张卷积操作，使得模型能够逐渐扩展其感受野，逐步吸收更广泛范围的上下文信息。这种逐层迭代的方式有效地增强了模型对文本信息的理解力，有助于更全面、准确地识别命名实体。

此技术的独特之处在于其高效的信息提取方式，不仅有助于避免参数爆炸，而且能够在相对较小的模型规模下获得更强大的性能。因此，通过引入迭代扩张卷积技术，命名实体识别模型在处理长距离上下文信息时表现出色。

2.6.3 RE_BGRU_2ATT

- 传统的循环神经网络（RNN）只考虑了历史信息，而双向 GRU 引入了正向和反向两个方向的 GRU 单元，使模型可以同时考虑上下文信息。正向 GRU 按照正序处理输入序列，而反向 GRU 按照逆序处理输入序列。通过将正向和反向的隐藏状态进行拼接或相加，双向 GRU 能够更全面地捕捉输入序列中的上下文信息，提高了模型对序列的建模能力。同时将原有的 LSTM 替换为 GRU，简化了模型结构，减少了参数数量，并且在某些任务上取得了与 LSTM 相当甚至更好的性能。

- 引入字级别注意力机制，针对每个字（字符）进行建模，将注意力权重分配给不同的字，以便模型更关注重要的字。通过计算每个字的注意力权重，模型可以根据其重要性调整每个字的表示贡献。字级别注意力机制使模型能够更细粒度地处理文本，从而提升了模型对文本细节的理解能力。

- 引入句子级别注意力，句子级别注意力机制基于字级别的表示，对每个句子进行加权组合，得到句子级别的表示。通过计算每个句子的注意力权重，模型可以根据句子的重要性对不同句子的表

示进行加权，从而更好地捕捉句子级别的语义信息。句子级别注意力机制使模型能够更好地理解整个文本的上下文关系和语义结构，提高了模型对文本整体含义的把握能力。

综上所述，本模型的创新点在于通过更全面地考虑上下文信息、更细粒度地处理文本和更好地捕捉句子级别语义，从而提高了模型对输入文本的建模能力和表示能力，进一步改善了文本分类任务的效果。这些创新点使得模型能够更好地理解和利用文本中的信息，提高了模型的性能和表现。

2.6.4 PL-maker

PL-Marker 主要对 NER 和 RE 中 span 的表征进行改进，在之前的工作中，有三种 span 表征方式：

- **T-Concat**：这种方式将 span 的 start 和 end token 的 representation 拼接起来作为 span 的 representation，也就是 PURE 中 NER 用的方式，其实很多模型都用的这种方式
- **Solid Marker（固定标记)**: 显式的在句子中的 span 前后插入两个 marker，如果是关系抽取，就在 subject span 和 object span 前后，但这种方式没法一次性处理多对 span，因为在句子中要塞入多对 marker 的话，不但句子结构会很乱，而且对于有交叉的 span 也不好处理。PURE-Full 就用的这种方式。
- **Levitated Marker（悬浮标记)**: 就是 PURE-Approx，这里不赘述了，目前仅有 PURE-Approx 中用了这种方式，效果相较于 PURE-Full 是有折扣的，论文中认为简单将悬浮标记放到句子后面的方式没有考虑多个 span 之间的关联。

该模型对 span 表征的建模采用的是后两种 Marker 的方式，针对单个 span 和多个 span 提出了两种标签打包的策略：

- 在做 NER 的时候提出了：Neighborhood-oriented Packing for span，就是将相邻的 span 的悬浮标记拼接在同一个样本里面；
- 在做 RE 的时候提出了：Subject-oriented Packing for span pair，就是将 subject span 用固定标记插入句子中，其对应的 Object span 们，用悬浮标记拼接在句子后面，放在一个样本里面；

2.7 实验总结

1. **姓名：**程子洋

   学号：2021301051114

   负责任务模块：

   - 资料论文查找
   - 数据爬取
   - 数据标注
   - 模型运行与评估
   - 实验报告撰写。

   完成情况：

   - 负责爬取推特数据 (约 3000 余条原始数据)
   - 标注近 500 条数据
   - 负责 NER_IDCNN、RE_BGRU_2ATT 模型所需数据的预处理工作
   - 负责 NER_IDCNN、RE_BGRU_2ATT 模型搭建与训练，并进行模型评估
   - 提出并尝试 PL-maker 模型，但因效果过差而放弃该模型的后续工作
   - 完成上述任务对应部分实验报告的撰写。

2. **姓名：**彭子恒

   学号：2021302091053

   负责任务模块：

   - 资料、论文查找
   - 数据标注
   - 数据处理
   - 模型运行与评估
   - 网页 demo 设计
   - 实验报告

   完成情况：

   - 在 google.scholar 上查找有关命名实体识别和关系抽取方面论文，并查找相关开源项目。
   - 标注推特数据 800 条，其中有效标注数据 400 条。
   - 整合全组标注数据，负责 CasRel 模型所需数据的预处理工作
   - 负责 CasRel 模型搭建与训练，并进行模型评估。
   - 参与 Pl-marker 模型的搭建与训练过程，后因效果差而放弃该模型后续工作。
   - 参与模型的前端展示
   - 完成上述任务对应部分实验报告撰写

3. **姓名：**王卓

   学号：2021302191791

   负责任务模块：

   - 资料、论文查找
   - 数据爬取
   - 数据标注
   - Pl-marker 模型数据处理
   - CasRel 模型运行与评估
   - 网页 demo 设计
   - 实验报告

   完成情况：

   - 资料、论文查找
     基于知乎的有关模型介绍，采用 CasRel 模型及 PURE 模型的升级模型 Pl-marker。
   - 数据爬取
     个人独立完成。
     采集 New York Times 与 BBC News 中文 (本次实验未使用) 数据共计约 1w2 条。
   - 数据标注
     个人独立完成。
     标注筛选后数据约 350/2000。
   - Pl-marker 模型数据处理
     与程子洋同学合作完成。
     完成对 2000 条标注数据的处理，即转换为 Pl-marker 模型输入数据的目标格式，但最终模型准确率不太理想，故放弃该模型。
   - CasRel 模型运行与评估
     与彭子恒同学合作完成。
     完成 CasRel 模型的代码实现、模型测试、加载与评估。
   - 网页 demo 展示
     个人独立完成。
     完成前端界面的设计、构建前后端数据传输交互核心框架、实现传输数据与 CasRel 模型的交互。
   - 实验报告
     撰写实验报告中个人独立完成及合作完成的相关部分内容。

4. **姓名：**邵嘉阳

   学号：2021302141070

   负责任务模块：

   - 资料、论文查找
   - 数据清洗
   - 数据标注
   - 模型运行与评估
   - 实验报告

   完成情况：

   - 对所有爬取的新闻数据进行清洗筛选，初步筛选出约 3000 条数据;
   - 完成 500 条新闻、超过三万字的数据标注工作和关系标注工作;
   - 尝试实现 PL-Marker 模型的模型搭建、训练、加载和评估工作，由于效果不理想而放弃;
   - 实现 RE_BGRU_2ATT 的模型搭建、训练、加载和评估工作;
   - 实现 RE_BGRU_2ATT 数据评估部分的可视化呈现和关系抽取的成果展示;
   - 完成实验报告的撰写;

5. **姓名：刘**琥

   学号：2021302121234

   负责任务模块：

   - 资料、论文查找
   - 数据标注
   - 模型运行
   - 实验报告

   完成情况：

   - 标注从纽约时报爬取的数据集部分。
   - PL-maker 模型根据论文和源码将其自带数据集复现成功。由于缺少中文训练数据集的规范格式，此模型对于本次实验效果不佳而放弃使用。
   - 根据自然语言处理课程的源码完成 BiLSTM+CRF 模型的代码实现、模型训练 (此方案被舍弃)。
   - 撰写实验报告中研究内容部分、数据集标注部分、模型介绍的 PL-maker 模型部分、创新性说明的 PL-maker 模型部分

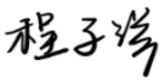
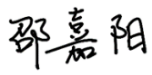
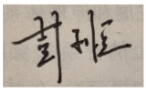
评语及评分（指导教师）

评语：

评　　分：

日　　期：

附件:

团队成员	数字签名
程子洋	
邵嘉阳	
王卓	
彭子恒	
刘虓	